

Testing Guide for Security Monitoring Dashboard

This guide will walk you through testing all features of your Security Monitoring Dashboard - both defensive monitoring and offensive security testing capabilities.

Quick Start Testing

1. Start the Dashboard

```
cd /home/ubuntu/security_dashboard
source .venv/bin/activate
python run.py
```

The server will start on `http://localhost:5000`

2. Access the Web Interface

Open your browser and navigate to:

- **Local VM:** `http://localhost:5000`
- **From your computer:** Check the preview URL (see below)

Default credentials:

- Username: `admin`
 - Password: `changeme`
-

Getting Your Preview URL

Since the dashboard is running on the Abacus AI VM, you can access it from your local browser using a preview URL:

```
# Get your preview URL
echo $PREVIEW_URL
```

This will give you a URL like `https://abc123.na110.preview.abacusai.app` that you can open in your local browser.

Testing Defensive Features

Test 1: Network Monitoring

What it does: Monitors all active network connections in real-time.

1. **View Network Activity:**

- Log in to the dashboard
- Go to **Overview** tab
- Observe the “Network Throughput” chart updating every 5 seconds
- Check “Active Connections” counter

2. **Generate Network Traffic:**

```
bash
# Open a new terminal
# Make some network connections to see them appear
curl https://google.com
curl https://github.com
wget https://www.abacus.ai -O /tmp/test.html
```

3. **Check Results:**

- Go to **Connections** tab
 - You should see your new connections listed
 - Filter by typing “google” or “github”
-

Test 2: Intrusion Detection System (IDS)

What it does: Monitors for suspicious login attempts and network intrusions.

Option A: Test with fail2ban (if installed)

1. **Check IDS Status:**

- Go to **IDS** tab in the dashboard
- You should see fail2ban jails (e.g., `sshd`)

2. **Simulate SSH Brute Force Attack:**

```
bash
# On your VPS, try to SSH with wrong password multiple times
# From another terminal or machine:
ssh wronguser@localhost # Enter wrong password 5+ times
```

3. **Check Results:**

- Refresh the **IDS** tab
- You should see the banned IP
- Check “Failed” and “Banned” counts increase

Option B: Test Built-in SSH Watcher

If fail2ban is not installed, the dashboard has a built-in SSH watcher:

1. **Check Alerts:**

- Go to **Alerts** tab
- Look for SSH brute-force alerts

2. View Logs:

```
bash
# Check alert log
tail -f logs/alerts.log
```

Test 3: Automatic IP Blocking

What it does: Automatically blocks IPs that make too many connections (flood protection).

1. Configure Threshold:

```
Edit config/config.yaml :
yaml
network:
  connection_threshold: 5 # Block IPs with >5 connections
```

2. Test Flood Detection:

```
bash
# Create multiple rapid connections from one IP
for i in {1..10}; do curl https://google.com & done
```

3. Check Results:

- Go to **Alerts** tab - you should see a “Connection flood” alert
 - The suspicious IP will be automatically blocked (if firewall enforce is enabled)
-

Test 4: Manual Firewall Management

What it does: Allows you to manually block/unblock IPs.

Test Blocking an IP:

1. Via Web Interface:

- Go to **Firewall** tab
- In the “Block an IP” section, enter: `1.2.3.4`
- Enter reason: “Testing block functionality”
- Click **Block IP**
- You should see it appear in the “Currently Blocked IPs” list

2. Via API (command line):

```
```bash
Block an IP
curl -X POST http://localhost:5000/api/firewall/block \
-H "Content-Type: application/json" \
-d '{"ip":"5.6.7.8","reason":"CLI test"}' \
-b cookies.txt

View blocked IPs
curl http://localhost:5000/api/firewall/status -b cookies.txt | jq
```
```

Test Unblocking:

1. Via Web Interface:

- In the “Currently Blocked IPs” list, click **Unblock** next to 1.2.3.4
- It should disappear from the list

2. Verify Firewall Rules:

```
```bash
If using UFW
sudo ufw status numbered

If using iptables
sudo iptables -L SECDASH_BLOCK -v -n
```
```

Test 5: Dry-Run Mode (Safe Testing)

What it does: Simulates firewall changes without actually applying them.

1. Enable Dry-Run Mode:

```
Edit config/config.yaml :
yaml
firewall:
  enforce: false # Dry-run mode
```

2. Restart the Dashboard:

```
bash
# Stop with Ctrl+C, then restart
python run.py
```

3. Try Blocking:

- Try blocking an IP via the Firewall tab
 - Check logs to see the simulated command:
- ```
bash
tail -f logs/app.log | grep "DRY-RUN"
```
- You'll see: [DRY-RUN] Would execute: sudo ufw ...
- 



## Testing Offensive Features

---

### Prerequisites: Enable Offensive Mode

#### 1. Edit Configuration:

```
```yaml
# config/config.yaml
offensive:
  enabled: true
  target_allowlist:
    - "127.0.0.1"
    - "192.168.1.0/24" # Your local network
```

- "scanme.nmap.org" # Legal test target
- ...

2. Restart Dashboard:

```
bash
python run.py
```

3. Verify Nmap is Installed:

```
bash
which nmap
# If not installed:
sudo apt-get install -y nmap
```

Test 6: Basic Port Scanning

What it does: Scans a target for open ports and services.

Via Web Interface:

1. Navigate to Scanner Tab:

- Click **Scanner** in the sidebar
- Read and acknowledge the warning banner

2. Run a Quick Scan:

- **Target:** scanme.nmap.org (legal test target)
- **Profile:** Select "Quick Scan"
- **Ports:** Leave default
- Check the **"I authorize this scan"** checkbox
- Click **Start Scan**

3. View Results:

- Wait 10-30 seconds
- The scan will appear in "Recent Scans" section
- Click **View Details** to see:
 - Open ports
 - Running services
 - Service versions
 - OS detection results

Via Command Line:

```
# Start a scan via API
curl -X POST http://localhost:5000/api/scan/start \
  -H "Content-Type: application/json" \
  -d '{
    "target": "scanme.nmap.org",
    "profile": "quick",
    "authorized": true
  }' \
  -b cookies.txt

# List recent scans
curl http://localhost:5000/api/scan/list -b cookies.txt | jq

# Get specific scan details (replace SCAN_ID)
curl http://localhost:5000/api/scan/<SCAN_ID> -b cookies.txt | jq
```

Test 7: Service Version Detection

What it does: Identifies software versions running on open ports.

1. Run Service Scan:

- Target: `scanme.nmap.org`
- Profile: **“Service Detection”**
- Ports: `22,80,443,8080`
- Authorize and start

2. Check Results:

- Look for service names and versions
- Example output:
 - Port 22: ssh (OpenSSH 7.9)
 - Port 80: http (Apache 2.4.41)

Test 8: Vulnerability Scanning

What it does: Runs Nmap NSE scripts to detect common vulnerabilities.

1. Run Vulnerability Scan:

- Target: `scanme.nmap.org`
- Profile: **“Vulnerability Scan”**
- Authorize and start
- **Note:** This takes 2-5 minutes

2. Check Results:

- Click **View Details** on the completed scan
- Look for the “NSE Script Results” section
- Common findings:
 - SSL/TLS vulnerabilities
 - Weak ciphers
 - HTTP methods

- Banner information

Test 9: OS Detection

What it does: Attempts to identify the target's operating system.

1. Run OS Detection Scan:

- Target: `scanme.nmap.org`
- Profile: **"OS Detection"**
- Authorize and start

2. Check Results:

- Look for "OS Guesses" in the scan details
- Example: `Linux 3.x - 4.x (95% confidence)`

Test 10: Host Discovery (Network Sweep)

What it does: Discovers all active hosts on a network.

 **Only use on networks you own!**

1. Add Your Network to Allowlist:

```
```yaml
config/config.yaml
offensive:
 target_allowlist:
 ◦ "192.168.1.0/24" # Your local network
```
```

2. Run Host Discovery:

- Target: `192.168.1.0/24` (your network)
- Profile: **"Host Discovery"**
- Authorize and start

3. Check Results:

- See all active hosts on your network
- Each host shows its IP and status

Test 11: Testing Authorization & Safety Features

What it does: Ensures you can't accidentally scan unauthorized targets.

Test 1: Scan Without Authorization

1. Try to start a scan **without** checking the authorization checkbox
2. **Expected result:** Error message "You must authorize this scan"

Test 2: Scan Blocked Target

1. Try to scan a target NOT in your allowlist (e.g., `google.com`)
2. **Expected result:** Error "Target not in allowlist"

Test 3: Scan With Denylist

1. Add to Configuration:

```
```yaml
offensive:
target_denylist:
 - "8.8.8.8" # Block Google DNS
```
```

2. Try to scan 8.8.8.8

3. Expected result: Error "Target in denylist"



Testing Alerts & Logging

Test 12: Alert System

1. Generate Some Alerts:

- Trigger connection flood (Test 3)
- Block an IP manually (Test 4)
- Run a vulnerability scan (Test 8)

2. View Alerts:

- Go to **Alerts** tab
- Filter by severity: **High, Medium, Low**
- Click **Ack** to acknowledge alerts

3. Check Alert Logs:

```
bash
tail -f logs/alerts.log
cat logs/alerts.log | jq '.'
```

Test 13: Audit Trail

What it does: Logs all security-critical actions.

1. Perform Various Actions:

- Log in/out
- Block/unblock IPs
- Run scans
- Change password

2. View Audit Log:

```
bash
tail -f logs/audit.log
cat logs/audit.log | jq '.'
```

3. Check for:

- "event": "login"
- "event": "ip_blocked"

- "event": "scan_complete"
 - Timestamps, actors, and details
-

Advanced Testing Scenarios

Scenario 1: Simulated Attack Response

Goal: Test the full defensive workflow.

1. Enable Auto-Blocking:

```
yaml
firewall:
  enforce: true
network:
  connection_threshold: 3
```

2. Simulate Attack:

```
bash
# Generate rapid connections
for i in {1..10}; do curl http://example.com & done
```

3. Observe Response:

- Check **Alerts** for flood detection
 - Check **Firewall** for auto-blocked IP
 - Check **Audit Log** for the block event
-

Scenario 2: Bug Bounty Recon Workflow

Goal: Test offensive capabilities for bug bounty.

1. Add Target to Allowlist:

```
``yaml
offensive:
  target_allowlist:
    - "target.example.com" # Your bug bounty target
``
```

2. Reconnaissance Steps:

- Run **Host Discovery** on target network
- Run **Quick Scan** on discovered hosts
- Run **Service Detection** on interesting ports
- Run **Vulnerability Scan** on web servers

3. Document Findings:

- Export scan results from UI
 - Check audit log for scan history
-

Scenario 3: Family Network Protection

Goal: Monitor your home network for threats.

1. Setup:

- Install on your home server/Raspberry Pi
- Configure to monitor local network
- Enable fail2ban for SSH protection

2. Monitor:

- Check **Overview** daily for unusual activity
- Review **Alerts** for security events
- Use **Connections** tab to see what devices are connecting

3. Block Threats:

- Manually block suspicious IPs
- Let auto-blocking handle flood attacks

Troubleshooting Tests

Test Not Working? Check These:

Firewall blocks not applying:

```
# Check if you have privileges
sudo -v

# Check firewall status
sudo ufw status # For UFW
sudo iptables -L # For iptables

# Check dashboard logs
tail -f logs/app.log
```

Scans failing:

```
# Verify nmap is installed
which nmap
nmap --version

# Check if target is in allowlist
cat config/config.yaml | grep -A5 target_allowlist

# Check scan logs
tail -f logs/app.log | grep scan
```

IDS not working:

```
# Check fail2ban status
sudo systemctl status fail2ban
sudo fail2ban-client status

# Check SSH log
sudo tail -f /var/log/auth.log
```

Dashboard not accessible:

```
# Check if server is running
ps aux | grep python | grep run.py

# Check port is listening
sudo netstat -tlnp | grep 5000

# Check firewall rules
sudo ufw status | grep 5000
```

**Testing Checklist**

Use this checklist to ensure all features are working:

Defensive Features:

- ☐ Dashboard loads successfully
- ☐ Login works with default credentials
- ☐ Password change works
- ☐ Network monitoring shows connections
- ☐ Network chart updates in real-time
- ☐ Can manually block an IP
- ☐ Can unblock an IP
- ☐ Blocked IPs appear in firewall list
- ☐ IDS tab shows jail status (if fail2ban installed)
- ☐ Alerts are generated and displayed
- ☐ Alert acknowledgment works
- ☐ Audit log records events
- ☐ System status shows correct info
- ☐ Dry-run mode works (logs but doesn't execute)

Offensive Features:

- ☐ Scanner tab is visible (when enabled)
- ☐ Scan profiles load correctly
- ☐ Authorization checkbox is required
- ☐ Can scan allowed targets
- ☐ Cannot scan blocked targets
- ☐ Quick scan completes successfully

- [] Service detection finds versions
 - [] OS detection works
 - [] Vulnerability scan finds issues
 - [] Scan results are parsed and displayed
 - [] Scan history shows recent scans
 - [] Can view detailed scan results
-

Educational Testing (For Teaching Others)

Demonstrating How Hackers Attack:

1. Show Port Scanning:

- Explain what port scanning is
- Run a scan on `scanme.nmap.org`
- Show how attackers find open services

2. Show Service Enumeration:

- Run service detection scan
- Explain how attackers identify software versions
- Discuss how this helps find vulnerabilities

3. Show Vulnerability Scanning:

- Run vulnerability scan
- Explain common vulnerabilities (SSL issues, weak ciphers)
- Discuss exploitation vs. responsible disclosure

4. Show Defensive Measures:

- Demonstrate IDS detecting attacks
 - Show automatic blocking in action
 - Explain firewall rules and protection
-

Important Safety Notes

1. Only scan targets you own or have permission to scan
 2. `scanme.nmap.org` is specifically provided by Nmap for testing
 3. Unauthorized scanning is illegal in most jurisdictions
 4. For bug bounty, always follow program rules
 5. Test in your own VPS/lab environment first
 6. Keep the dashboard behind a firewall/VPN
 7. Change default passwords immediately
 8. Review logs regularly for suspicious activity
-

Next Steps

After testing:

1. Read the Documentation:

- docs/DEFENSIVE.md - Defensive operations guide
- docs/OFFENSIVE.md - Offensive testing guide
- docs/BEST_PRACTICES.md - Security best practices

2. Secure Your Installation:

- Change default password
- Generate strong secret key
- Set up HTTPS with reverse proxy
- Restrict network access

3. Customize Configuration:

- Adjust thresholds for your needs
- Configure webhook alerts
- Set up your allow/deny lists
- Enable/disable features as needed

Getting Help

If you encounter issues:

1. Check logs:

```
bash
tail -f logs/app.log
tail -f logs/alerts.log
tail -f logs/audit.log
```

2. Enable debug mode:

```
yaml
server:
  debug: true
```

3. Check configuration:

```
bash
cat config/config.yaml
```

4. Verify prerequisites:

```
bash
./scripts/setup.sh --tools
```

Happy Testing! Stay Safe and Hack Ethically! 